

Structured Machine Learning – Project 1

Comparing Graph Neural Networks for Molecular Property Prediction

Assessment

To pass a project assignment you must:

1. Complete the code
2. Complete a report report addressing the questions outlined below

You must include details about the use of aids – Read ‘Use of aids’ section on canvas syllabus before starting. Submissions missing this will not be accepted.

You can submit a PDF render of your working notebook as your report

Project description and motivation

In this project, you will perform regression and classification analyses for molecular graphs. You will analyse two different architectures and two different data-sets and investigate what factors limit building a predictive model.

Last week, we talked about experimental observables, O , which are expectations with of some observable function $o(\cdot) : \Omega \rightarrow \mathbb{R}$ with respect to a Boltzmann distribution,

$$O_M = \mathbb{E}_{\mathbf{x} \sim p_M}[o(\mathbf{x})] = \int o(\mathbf{x}) p_M(\mathbf{x}) d\mathbf{x} \quad (0.1)$$

where the Boltzmann density is given by

$$p_M(\mathbf{x}) \triangleq \mathcal{Z}_\beta^{-1} \exp(-\beta u(\mathbf{x})), \quad \mathbf{x} \in \Omega \subset \mathbb{R}^d \quad (0.2)$$

for a given molecule M .

We have seen that sampling this density is very challenging, and for certain experiments, e.g. physiological data, first principles simulations are currently completely impractical. An alternative strategy is to use large volumes of data together with basic symmetries of molecular graphs to build predictive models which approximate the observables directly from the chemical compound.

Specifics

In this project, you will train invariant regression and classification models. Your model will take as input a set of node features and edge indices and predict a real scalar (regression) or a 'probability' (classification). For regression tasks you should use the mean-squared error loss (MSELoss in PyTorch) and for classification you should use the binary cross entropy (BCELoss in PyTorch).

Data You will be looking at a subset of the MoleculeNet benchmark [3], made conveniently available as part of the torch_geometric library:

https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.MoleculeNet.html

You should focus on the datasets ESOL (regression) and BBBP (binary classification). The first dataset is a small set of solubility measurements for molecules. These measurements, put simply, reflect how much a molecule favors being in water over being in a vacuum, from a thermodynamic perspective. The other dataset gives an indication whether a molecule can pass over blood-brain barrier, an important property for the development of pharmaceuticals targeting the central nervous system. *Hint: filter out graphs with fewer than two nodes, sometimes pre-processing fails silently, and it gives rise to invalid molecules in the dataset.*

Each Data object contains the following attributes:

x Node features

edge_index Index pairs for all edges in molecular graph

edge_attr Edge attributes for all edges (we do not use these)

smiles SMILES string (we only use this for stratification)

y Target value

Data splits: Data-splitting tools are provided to ensure the training, test, and validation splits are stratified according to chemical scaffold. Such scaffold splitting is important, as molecules with similar scaffolds tend to have similar properties, and we want to avoid over-fitting.

Code You are provided with a minimalistic code framework in a Jupyter notebook. You can find the notebook on the assignment homepage. The notebook contains some hints about how to structure your code and solve more difficult parts of the assignment, so be sure to check it carefully.

Optional tip: I recommend using Weights and Biases (<https://wandb.ai/>) to manage your training jobs. Wandb can help you keep track of your experiments and hyper-parameters, and export data for tables. It is free for academic and educational use.

Tasks

Build `torch.nn.Modules` for classification and regression which wraps a `torch-geometric` model, for example the Graph Attention Network (GAT in `pytorch-geometric`) or the Graph Convolutional Neural net (GCN). The node-features of the molecular graphs are integers and should be turned into embeddings, use for example `torch.nn.Embedding`.

In your report, briefly outline what you did. You will mostly use code from libraries so there is no reason to explain the individual components at this point. We will analyse them later.

Test your code! – If it works, move on.

Set up a general training script, a suggested scaffold is provided, use this to run your experiments.

You should train your models in at least 3 replicates (using different seeds for each model) with random weight initialization. Then use these to estimate errors of your reported values.

Hint: expected timings to train a model is a few minutes.

Establish baseline models for each task Build a simple base-line model, use a DeepSet architecture (ignoring the graph structure), use a single hidden layer in the two MLP models with 64 hidden nodes. Report mean squared errors and Pearson correlation coefficients (for ESOL) and the area under the receiver operating characteristic (for BBBP). Report these values for training, test, and validation sets separately. Annotate with number of molecules (datapoints) in each data split.

Compare GAT and GCN Keeping all other things equal train models for regression and classification, one with GAT [2] and one with GCN [1].

Consult the two original papers. Identify the differences between the methods, make a plan to compare to test these differences with the two datasets you have available. Describe the plan in your report and execute it. Be sure to present results in a way which is comparable to your baseline.

It is important that you spend some time reflecting on the results you get here. Think about the key concepts we have discussed in the course including: domains, errors, symmetries, and the quality of the data.

References

- [1] KIPF, T. N., AND WELLING, M. Semi-supervised classification with graph convolutional networks, 2016.
- [2] VELIČKOVIĆ, P., CUCURULL, G., CASANOVA, A., ROMERO, A., LIÒ, P., AND BENGIO, Y. Graph attention networks, 2018.

- [3] WU, Z., RAMSUNDAR, B., FEINBERG, E. N., GOMES, J., GENIESSE, C., PAPPU, A. S., LESWING, K., AND PANDE, V. Moleculenet: A benchmark for molecular machine learning, 2017.